Title: Requirements Engineering-Driven Intelligent Software System with ML-Based Inconsistency Detection, Prediction, and Performance Evolution

Introduction

Requirements Engineering (RE) is a fundamental phase of software development. It determines what a software system should achieve and how its expected behavior should be defined. The requirements engineering process consists of five activities: requirements gathering and analysis, requirements specification, verification and validation, and requirements managements. The quality of requirements has a direct influence on software development because inaccurate, ambiguous, incomplete, or inconsistent requirements can damage and hamper software development throughout the cycle. Traditional requirements engineering activities depend heavily on manual analysis by engineers and developers. When a project contains many requirements, manual analysis becomes time-consuming and may introduce human error. For this reason, researchers have increasingly explored automated methods that can support more accurate and consistent requirements analysis [22].

Recent developments in Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), and Natural Language Processing (NLP) have made it possible to automate many requirements engineering tasks. Researchers have used these technologies for requirements classification, quality checking, ambiguity detection, prioritization, dependency analysis, elicitation, traceability, and requirements management. Earlier studies mainly relied on traditional machine learning methods such as Support Vector Machines, Naive Bayes, and decision trees for automatic requirements classification. These methods often use text representation techniques such as Bag-of-Words and Term Frequency-Inverse Document Frequency. Although they produced useful results, their performance often depended on careful feature selection and the quality of labeled data [12].

After the development of deep learning and transformer-based language models, it has enhanced the capability of automated systems to extract relevant information from requirements. There are some models like LSTM, Bi-LSTM, GRU, BERT, RoBERTa, and others transformer architectures have proved strong performance in various requirements analysis tasks. Research has identified some challenges like limited datasets, class imbalance, dataset diversity, domain generalization, computational costs, and inconsistent evaluation methods. The repeated use of benchmark datasets like PROMISE has also raised concerns regarding whether models can effectively generalize to new software domains.

Interpretability and reliability are also important concerns in AI-based requirements analysis. Explainable AI methods such as LIME, SHAP, and attention-based visualization can help users understand the factors that influence model predictions [21]. However, explanations may become unstable when requirements are heavily rewritten or semantically changed, which limits how confidently they can be trusted [5]. More recently, Large Language Models (LLMs) and generative AI have expanded automated RE into areas such as user-story generation, elicitation, and analysis.

At the same time, hallucination, reproducibility, output inconsistency, and limited interpretability remain important concerns [4].

Despite significant progress, existing research indicates that many AI-based approaches still focus on individual requirements engineering tasks, rely heavily on benchmark datasets, and offer limited industrial validation. In addition, relatively less attention is paid to the reliability of explanations and credible evaluation compared to predictive performance. These limitations create a need for AI-based requirement analysis methods that can combine reliable assessment and interpretable decision support with accurate predictions. Addressing these issues could be able to contribute to the development of intelligent systems that are more reliable and practically useful for software requirements engineering.

Literature Review

Software Requirements Engineering (RE) has increasingly adopted Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), Natural Language Processing (NLP), and Large Language Models (LLMs) to reduce manual effort and improve requirements analysis. Recent research shows a clear shift from traditional classification methods toward adaptive data augmentation, transformer-based models, explainable AI, and generative approaches. At the same time, problems such as class imbalance, limited datasets, weak generalization, interpretability, and lack of industrial validation continue to affect practical use.

Aldhafer et al. [1] proposed adaptive data augmentation for software requirements classification using deep learning. Their method combined Weighted Label-Based Data Augmentation with dictionary-based and transformer-based semantic augmentation in a BiGRU pipeline. The approach mainly targeted underrepresented requirement classes. Their adaptive WLLB method improved the F1-score of minority classes by up to 9.3% and increased macro F1 by 6.7% on the PROMISE ten-class dataset. Transformer-based augmentation achieved 84.1% F1 on the PROMISE multilabel dataset, while dictionary-based augmentation improved EHR multilabel F1 by 5.2%. However, the method depended on carefully prepared domain dictionaries, and uniform augmentation could increase majority-class dominance.

Sami et al. [2] introduced a multi-agent LLM framework for automated requirements analysis. Their system assigned roles such as Product Owner, Developer, QA, and Manager to separate agents and evaluated GPT-3.5 Turbo, GPT-4o, LLaMA 3.3-70B, and Mistral-Nemo for user-story generation and prioritization. LLaMA 3.3 achieved the highest average semantic similarity score of 0.50, while GPT-4o provided broad feature coverage. Although the framework supported requirements generation and prioritization, some outputs were repetitive or unclear. Rankings did not always match expert judgment, and results varied across repeated runs.

Zhao et al. [3] explored AI-supported software quality assurance using PyTest-BDD. Their framework applied multilingual BERT to translate natural-language requirements into Gherkin scenarios and automated tests. This reduced some manual effort involved in converting

requirements into testing specifications. The method also considered reinforcement learning for test prioritization. However, multilingual differences, domain-specific terminology, maintenance effort, and incorrect test execution could still require manual review.

Cheng et al. [4] reviewed 238 studies on generative AI in requirements engineering. GPT-based models appeared in 67.3% of the surveyed studies. Requirements analysis represented around 30% of the research, while elicitation accounted for 22.1% and requirements management only 6.8%. More than 90% of the proposed approaches were still early-stage prototypes, and only around 1.3% had reached production. Major concerns included hallucination, reproducibility, limited interpretability, dependence on proprietary models, and the lack of standard evaluation benchmarks.

Petrillo et al. [5] studied explainable security requirements classification using BERT, DistilBERT, RoBERTa, XLNet, and Few-Shot SetFit. RoBERTa-base achieved 0.90 accuracy and an F1-score of 0.86, while few-shot RoBERTa reached 0.92 accuracy and 0.97 F1 on the normal dataset. Attention visualization was used to make predictions easier to understand. Contextual word augmentation also improved performance, although strong semantic rephrasing still reduced classification reliability.

Alem et al. [6] examined multi-label software requirement smell classification using Bi-LSTM, LSTM, and GRU with ELMo and Word2Vec embeddings. Bi-LSTM with ELMo achieved the highest macro-average F1-score of 90.3% across 11 smell categories. The study showed that contextual representations can help identify multiple quality problems in requirements. However, the binary relevance method treated each smell independently and did not capture relationships among smell categories. Informal language in app-store reviews also affected performance.

Zhao and Alhoshan [7] reviewed machine learning approaches for requirements classification, including traditional ML, deep learning, and pretrained language models. Their work covered SVM, Naïve Bayes, CNN, RNN, and BERT-based methods for functional and non-functional requirements classification. BERT-based methods often achieved strong F1-scores, commonly above 80% to 90%. However, class imbalance, limited benchmark datasets, and difficulties in applying experimental models in industrial environments remained major concerns.

Romao et al. [8] proposed a requirements-focused agile approach for ML-enabled systems. Their method used ML-specific requirements, separate software and ML backlogs, and a Minimal Viable Model. Developers responded positively to the separation of software-related and ML-related activities. However, the study was evaluated in only one industrial case, which limited the generalizability of the findings.

Ezzini et al. [9] developed the Paska tool for detecting requirement smells in natural-language requirements. The tool combined rule-based NLP, morphological analysis, and pattern matching and achieved around 89% overall precision on financial software requirements. Its performance decreased when it encountered conditional scenarios not represented in the original data. Since the

evaluation mainly focused on the financial domain, the results may not apply equally well to other software domains.

Hassan et al. [10] conducted a systematic mapping study of machine learning methods used in requirements engineering. Decision Tree, Support Vector Machine, Naïve Bayes, K-Nearest Neighbors, and Random Forest were among the most frequently used supervised algorithms. Although these methods were widely applied, the study reported a lack of clear guidance for choosing the most suitable algorithm for a particular RE task.

Akram et al. [11] reviewed recommender-system approaches for requirements elicitation. Collaborative filtering, content-based filtering, and hybrid methods were used to help stakeholders identify missing requirements and prioritize feedback. However, cold-start problems, sparse stakeholder data, and limited industrial testing reduced practical effectiveness. Kaur and Kaur [12] reviewed 61 studies on AI-based requirements classification and found that BERT, RoBERTa, and DistilBERT generally performed better than many traditional ML and earlier deep learning methods. They also identified repeated dependence on the PROMISE dataset, inconsistent evaluation metrics, limited reproducibility, and poor interpretability.

Yucalar [13] evaluated 20 algorithms on a Turkish Software Requirements Classification dataset containing 4,600 requirements. Fine-tuned BERTurk achieved the highest performance with a 95.4% F1-score and an AUC of 0.983. CNN achieved 93.7% F1, while Multinomial Naïve Bayes achieved 92.0%. The results demonstrated the strength of pretrained language models, although the dataset was imbalanced and limited to Turkish-language requirements.

Muhamad et al. [14] proposed a method for identifying fault-prone Software Requirements Specifications using CNN, RNN, LSTM, word embeddings, and boosting techniques. Their Ambiguity Classification Model achieved 0.9907 accuracy, while the fault-prone detection model achieved 0.9750 accuracy. Despite the strong results, the approach required human involvement during some stages and introduced additional computational cost. Raczkowska-Gzowska and Walkowiak-Gall [15] examined what a good Software Requirements Specification should contain and identified functional behavior, interface constraints, non-functional quality attributes, and acceptance criteria as important elements. However, the findings were based on practitioner opinions and could be affected by response bias.

Xu et al. [16] studied machine learning approaches for requirements management and found that supervised learning accounted for 64.2% of the reviewed studies. Unsupervised learning, semi-supervised learning, deep learning, active learning, and reinforcement learning appeared less frequently. Their study also reported a shortage of standard ground-truth datasets and limited explainability.

Rahimi and Saleem [17] reviewed deep learning methods for classifying software system attributes. CNN and RNN models achieved around 92% to 93.4% accuracy in some multiclass non-functional requirements classification tasks. However, performance depended strongly on

dataset quality and word representation. Martínez-Fernández et al. [18] surveyed software engineering practices for AI-based systems and found that testing and quality assurance received more attention than requirements engineering. Their findings suggested that RE practices for AI-based systems still require further development and evaluation.

Cheligeer et al. [19] reviewed 86 studies related to machine learning in requirements elicitation. Their review covered Naïve Bayes, SVM, Decision Tree, Random Forest, CNN, RNN, LDA, and BTM, along with Bag-of-Words, TF-IDF, Word2Vec, GloVe, and BERT. Text mining and classification were widely used, but limited industrial datasets and difficulties in handling changing requirements remained major challenges.

Giray [20] reviewed software engineering research for ML-enabled systems and found that testing and software quality received more attention than requirements engineering. This showed a need for stronger RE practices for systems that include machine learning components. Tantithamthavorn et al. [21] studied explainable AI methods such as LIME, SHAP, AutoSpearman, and SQAPlanner. Their findings showed that these methods can help developers understand model predictions and identify influential features, although correlated variables and computational cost can reduce explanation quality.

Finally, Zhao et al. [22] conducted a systematic mapping study of 404 publications on NLP for requirements engineering. Requirements analysis and elicitation were among the most frequently studied areas. Their review included symbolic, statistical, machine learning, and deep learning approaches. However, many tools remained academic prototypes, while limited shared datasets, source code, and industrial evaluation continued to restrict practical adoption.

Overall, the literature shows a clear progression from traditional machine learning and NLP toward deep learning, pretrained transformers, adaptive data augmentation, explainable AI, generative AI, and multi-agent LLM systems. Although classification performance has improved, several limitations remain, including class imbalance, outdated or narrow datasets, weak cross-domain generalization, limited explainability, inconsistent evaluation, and low industrial adoption. Future research should focus on diverse datasets, augmentation methods that preserve requirement meaning, more reliable explanations, consistent evaluation procedures, and validation using real industrial requirements.

References

[1] O. Q. Aldhafer, H. A. Alsadiq, and M. Alshaboti, “Adaptive Data Augmentation Techniques for Software Requirements Classification Using Deep Learning,” *IEEE Access*, vol. 14, pp. 1–15, 2026.

[2] M. A. Sami, Z. Zhang, M. Waseem, K.-K. Kemell, Z. Rasheed, T. Herda, M. T. Hasan, J. Rasku, and P. Abrahamsson, “A Multi-agent LLM System for Automated Requirements Analysis: A Study on User Story Generation and Prioritization,” in *Proc. 51st Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, LNCS, vol. 16082, Springer, pp. 178–187, 2026.

[3] X. Zhao, H. Wang, J. Ding, Z. Hu, Q. Tian, and Y. Wang, “Augmenting software quality assurance with AI and automation using PyTest-BDD,” *Automated Software Engineering*, vol. 33, art. no. 23, pp. 1–44, 2026.

[4] H. Cheng *et al*., “Generative AI for Requirements Engineering: A Systematic Literature Review,” *Software: Practice and Experience*, pp. 1–32, 2025.

[5] L. Petrillo, F. Martinelli, A. Santone, and P. Mercaldo, “Explainable Security Requirements Classification Through Transformer Models,” *Future Internet*, vol. 17, no. 1, art. no. 15, pp. 1–27, 2025.

[6] A. L. Alem, K. K. Gebretsadik, S. A. Mengistie, and M. F. Admas, “Multi-label software requirement smells classification using deep learning,” *Scientific Reports*, vol. 15, art. no. 5761, pp. 1–16, 2025.

[7] L. Zhao and W. Alhoshan, “Machine Learning for Requirements Classification,” in *Machine Learning for Requirements Engineering*. Springer, pp. 19–45, 2025.

[8] L. Romao, L. Xavier, J. C. Araújo, M. C. Araújo, A. Rodrigues, and M. Kalinowski, “Applying a Requirements-Focused Agile Management Approach for Machine Learning-Enabled Systems,” in *Proc. Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, pp. 1–12, 2025.

[9] S. Ezzini, S. Abualhaija, C. Arora, M. Sabetzadeh, and L. C. Briand, “Automated Smell Detection and Recommendation in Natural Language Requirements,” *IEEE Transactions on Software Engineering*, pp. 1–18, 2024.

[10] S. Hassan, Q. Li, K. Aurangzeb, A. Yasin, J. A. Khan, and M. S. Anwar, “A systematic mapping to investigate the application of machine learning techniques in requirement engineering activities,” *CAAI Transactions on Intelligence Technology*, vol. 9, no. 6, pp. 1412–1434, 2024.

[11] R. Akram *et al*., “Recommendation systems-based software requirements elicitation process: A systematic literature review,” *Journal of Engineering and Applied Science*, vol. 71, art. no. 29, pp. 1–21, 2024.

[12] K. Kaur and P. Kaur, “The application of AI techniques in requirements classification: A systematic literature review,” *Automated Software Engineering*, vol. 31, pp. 1–48, 2024.

[13] F. Yucalar, “Developing an Advanced Software Requirements Classification Model Using BERT: An Empirical Evaluation Study on Newly Generated Turkish Data,” *Applied Sciences*, vol. 13, no. 20, art. no. 11127, pp. 1–22, 2023.

[14] F. N. J. Muhamad, S. H. A. Hamid, H. Subramaniam, R. A. Rashid, and F. Fahmi, “Fault-Prone Software Requirements Specification Detection Using Ensemble Learning for Edge/Cloud Applications,” *Applied Sciences*, vol. 13, no. 14, art. no. 8368, pp. 1–33, 2023.

[15] K. Raczkowska-Gzowska and A. Walkowiak-Gall, “What Should a Good Software Requirements Specification Include? Results of a Survey,” in *Proc. Int. Conf. Software Engineering*, pp. 71–81, 2023.

[16] C. Xu, Y. Li, B. Wang, and S. Dong, “A systematic mapping study on machine learning methodologies for requirements management,” *IET Software*, vol. 17, no. 4, pp. 405–423, 2022.

[17] N. Rahimi and N. N. Saleem, “Classification of Software Systems attributes based on quality factors using deep learning techniques: A Review,” *Journal of Education and Science*, vol. 31, no. 3, pp. 66–90, 2022.

[18] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, and S. Wagner, “Software Engineering for AI-Based Systems: A Survey,” *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 2, art. no. 37e, pp. 1–59, 2022.

[19] C. Cheligeer, J. Huang, G. Wu, N. Bhuiyan, Y. Xu, and Y. Zeng, “Machine learning in requirements elicitation: A literature review,” *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, vol. 36, art. no. e12, pp. 1–18, 2022.

[20] G. Giray, “A software engineering perspective on engineering machine learning systems: State of the art and challenges,” *The Journal of Systems & Software*, vol. 180, art. no. 111031, pp. 1–21, 2021.

[21] C. Tantithamthavorn, J. Jiarpakdee, and J. Grundy, “Explainable AI for Software Engineering,” *IEEE Software*, vol. 38, no. 4, pp. 115–120, 2021.

[22] L. Zhao, W. Alhoshan, A. Ferrari, K. Letsholo, M. Pratt, and N. Bencomo, “Natural Language Processing for Requirements Engineering: A Systematic Mapping Study,” *ACM Computing Surveys*, vol. 54, no. 3, art. no. 55, pp. 1–41, 2021.